

Assignment 2: CUDA Renderer

Group Members

Behroz Karim – [REDACTED]
Hasnain Ajmal – [REDACTED]
Hugo Pérez Muñoz – [REDACTED]
Talha Rizwan – [REDACTED]
Max Nummila – [REDACTED]

May 2026

1 System Information:

Property	Value
Architecture	x86_64
CPU	13th Gen Intel Core i5-13420H (Raptor Lake-H)
Cores	6 (12 hardware threads, 2 threads per core)
Max CPU clock	4.6 GHz turbo
SIMD Support	AVX2 (8-wide vector instructions for single-precision float)
Total Hardware Threads	12
GPU	NVIDIA GeForce RTX 3050 6 GB Laptop GPU
GPU architecture	Ampere, SM 8.6, 20 SMs
GPU memory	6 GB GDDR6, 168 GB/s peak bandwidth
GPU host link	PCIe Gen4 ×4
Host RAM	24 GB DDR5-5200 SODIMM (WSL2 sees 11 GB by default)
OS	Ubuntu 22.04 in WSL2 on Windows 11
NVIDIA driver	596.36
CUDA toolkit	12.4
Host compiler	GCC 11

2 Part 1: CUDA SAXPY

Measured results (default N = 100,000,000, 3 runs)

Run	Total time (memcpy + kernel + memcpy)	Kernel Only
1	----- 252.3 ms - 4.43 GB/s	----- 25.20 ms - 44.3 GB/s
2	----- 237.6 ms - 4.70 GB/s	----- 10.07 ms - 111.0 GB/s
3	----- 232.7 ms - 4.80 GB/s	----- 10.16 ms - 110.0 GB/s

The first run is slower than the others probably because CUDA does some one-time setup work the very first time a kernel is launched (context creation etc). That's why in our discussion below we ignore the 1st run and discuss based on results from runs 2 and 3.

For comparison, our Assignment 1 CPU SAXPY numbers on the same machine were:

[saxpy ispc]:	[9.158] ms	[32.542] GB/s	[4.368] GFLOPS
[saxpy task ispc]:	[6.328] ms	[47.097] GB/s	[6.321] GFLOPS
(1.45x speedup from use of tasks)			

2.1 Question 1: How does CUDA SAXPY compare to the CPU version?

It depends on which timer we look at:

Kernel-only, the GPU is faster: ~ 110 GB/s vs. ~ 47 GB/s for the best CPU version. That is about $2.3\times$ speedup, which makes sense because the GPU's GDDR6 memory has a peak bandwidth of ~ 168 GB/s while the laptop's RAM peaks at around ~ 83 GB/s in dual-channel mode. So the GPU has roughly $2\times$ the raw memory bandwidth available.

End-to-end (total), the GPU is actually slower: 4.8 GB/s vs. 47 GB/s on the CPU. That is roughly $10\times$ slower than the CPU.

The reason why GPU is slower in this case is simple. SAXPY performs very little actual math per byte (it performs a single multiply-add operation per pair of inputs). On the GPU we have to copy the input arrays over PCIe, run the tiny kernel, and copy the result back over PCIe. The PCIe transfers take far longer than the kernel itself, and the CPU does not pay this cost because the data is already in main memory. So for a one execution of SAXPY, the GPU is not a good fit. Thus, the use of SAXPY on the GPU is only justified if the data is already resident in GPU memory and will be used repeatedly in multiple executions of SAXPY.

2.2 Question 2: Why are the two timers so different, and do the bandwidths make sense?

The two timers differ by roughly $23\times$ (~ 232 ms total vs. ~ 10 ms kernel). In other words, the kernel itself is less than 5% of the total time and the rest of the time is spent in the three cudaMemcpy calls that move 1.2 GB of data over PCIe (x and y to the GPU, and results back).

These numbers actually make sense.

Kernel-only ≈ 110 GB/s is bounded by the GPU's GDDR6 memory bandwidth. The RTX 3050 6 GB Laptop GPU has a peak of about 168 GB/s, so we are hitting roughly 65% of peak. For a streaming kernel like SAXPY this looks normal as every load and store goes to DRAM and there's no cache reuse to exploit.

Total ≈ 4.8 GB/s is essentially a measurement of the PCIe link, not the GPU. The mobile RTX 3050 connects over PCIe Gen4 $\times 4$, with a theoretical peak of about 8 GB/s per direction. Real transfers from regular host memory typically reach 60–80% of that peak, which matches our 4.8 GB/s very well.

So the kernel-only timer is limited by GPU memory bandwidth, and the total timer is limited by PCIe bandwidth. The huge gap between them is just the ratio of those two limits, and our numbers are roughly consistent with both.

3 Part 2: CUDA Circle Renderer

We ran the code on the same machine that we described above [1](#)

On top of the existing starter code we added two new functions in the file `cudaRender.cu`. `shadePixelLocal` which is a modified version of `shadePixel`. A new kernel `kernelRenderTile` which is used as a replacement of the original `kernelRenderCircles`. To use our new functions we made the necessary changes in `CudaRender::render`.

3.1 Score Table

The table below is the output of the official `./checker.py`. Because the supplied `render_ref_x86` binary requires `glibc ≥ 2.38` and a `libstdc++` from `gcc-13`, I was unable to run it directly on **Ubuntu 22.04**, so I ran the full grader inside an `"nvidia/cuda:12.8.2-runtime-ubuntu24.04"` Docker container with `-gpus all` (the same physical RTX 3050 is exposed to the container via the NVIDIA Container Toolkit). All measured times are therefore on the same hardware mentioned above.

Scene Name	Ref Time (T_ref)	Your Time (T)	Score
rgb	0.588	0.6353	9
rand10k	5.3525	5.1496	9
rand100k	50.2157	47.501	9
pattern	0.7986	0.7942	9
snowsingle	32.4836	33.6809	9
biglittle	28.6526	23.4492	9
rand1M	328.2548	336.2399	9
micro2M	610.7057	629.8643	9
Total score:			72/72

All 8 scenes pass correctness and meet the $T \leq 1.20 \cdot T_{\text{ref}}$ threshold for full performance credit. We are faster than the reference on `rand10k`, `rand100k`, `pattern`, and `biglittle`, and within ~3–8% on the rest.

3.2 Decomposition

The starter code implements the parallelism in the wrong way. It gives **one CUDA thread per circle**. That means many threads can try to write to the same pixel at the same time, which is why the starter image has those horizontal streaks. We changed it and took a total opposite approach. Instead of one thread per circle, we use **one CUDA block per 16 x 16 pixel tile** and **one thread per pixel inside the tile** (so 256 threads per block).

For the graded image size (1024 x 1024), this gives $64 \times 64 = 4096$ blocks, each with 256 threads.

Each block then loops through the input circles in batches of 256. For each batch

- Every thread loads its circle and checks whether the circle overlaps the block's tile. To check the overlapping we use `circleInBoxConservative` from `circleBoxTest.cu.inl`.

- We use the shared-memory exclusive scan from `exclusiveScan.cu_inl` to compact the circles that survived the test into a per-tile list. The scan is stable, so the input order is preserved.
- Each pixel-thread then walks the per-tile list and applies each surviving circle to its pixel, in order.

This is how we get both atomicity and order.

Atomicity: Each pixel has exactly one owning thread for the entire frame. The pixel value is kept in a local variable (a register) and written back to global memory only once, at the end. No other thread can ever write to that pixel, so the read–blend–write does not need any locks or atomic operations.

Order: Circles are processed in input order (batches go in order, the scan is stable, and the apply loop walks the list left to right). So the order in which a given pixel sees circle contributions is exactly the input circle order.

Here’s the summary of how we decomposed or split the work:

- **Tile size:** 16×16 pixels.

- **Grid:**

$$\left(\frac{W}{16}\right) \times \left(\frac{H}{16}\right) \quad \text{blocks}$$

blocks For the graded image size (1024×1024), that is

$$64 \times 64 = 4096 \quad \text{blocks}$$

- **Block:**

$$16 \times 16 = 256 \quad \text{threads}$$

laid out as a 2-D `dim3(16, 16)`.

- **Linear thread index inside the block:**

$$\text{linearIdx} = \text{threadIdx.y} * \text{blockDim.x} + \text{threadIdx.x}$$

This layout is required by `sharedMemExclusiveScan` so that every group of 32 consecutive linear indices maps to a single warp.

- **Per-thread work:** thread `linearIdx` owns exactly one pixel of the tile and, in each circle batch, also tests `circle batch * 256 + linearIdx` against the tile bounding box.

- **Warps within a block:**

$$\frac{256}{32} = 8 \quad \text{wraps}$$

3.3 Synchronization

We do not use any **global synchronization, atomics, and locks** anywhere in the renderer. The only synchronization that we do is `__syncthreads()` inside each block, and it occurs three times per batch:

1. After every thread writes its 0/1 “does it overlap?” flag, before the scan reads it.

2. After the scan finishes, before threads read the destination index where they should put their circle.
3. After the surviving circles are written into the per-tile list, before the apply loop reads them.

These are cheap because they only synchronize the 256 threads of one block, not the whole grid. The only kernel-level synchronization is one `cudaDeviceSynchronize()` at the end of `render()`, so the host knows the kernel is done before it copies the image back.

3.4 Bandwidth Reduction

The renderer is bandwidth-bound which means every batch reads position and radius of 256 circles from global memory. So we tried to reduce the global memory traffic wherever we could:

- **Keep the pixel in a register.** The starter `shadePixel` that we had, it did 4 floats of read + 4 floats of write per circle per pixel, which would dominate everything. We changed it. In our implementation the pixel value is loaded from global memory once at the start of the kernel and written back once at the end. In between, even if thousands of circles contribute to it, all the blending happens in a register.
- **Cull circles per tile.** Every batch starts with a per-thread `circleInBoxConservative` check against the tile’s bounding box. For scenes with small circles like `rand10k`, `rand100k`, `rand1M`, `micro2M`, and `snowsingle`, most circles do not touch most tiles, so this gets done the majority of the work cheaply.
- **Store surviving circle data into shared memory.** After the scan, we write each surviving circle’s position (xyz) and radius into a `float4` in the shared memory. The apply loop reads from shared memory instead of going back to global memory. Since all 256 pixels of the tile read each surviving circle once, this gives us $256\times$ reuse of one shared-memory load.
- **No atomics anywhere.** We didn’t use `atomicCAS`, `atomicAdd`, or any locking because every pixel had only one owner. All memory accesses are plain reads and writes.

3.5 Arriving at the Solution

We tried two other approaches before the final one:

First idea was to keep the original “one thread per circle” design but put an `atomicCAS` lock around the read, blend, and write. It was easy to code but it was very slow because in this case every busy pixel becomes a bottleneck because all the threads touching it are forced to take turns. It also did not fix the order requirement and was requiring extra logic to ensure that circle 1 takes the lock before circle 2. That’s why we didn’t use this approach and started to find a suitable solution.

Our second idea was that each thread is a pixel, and it loops through all the circles checking for overlaps. This was actually correct because in this case atomicity and order were easy since each thread was a pixel. It was also fast on small scenes but for `rand1M` and `micro2M` it was much slower than the reference because every pixel read every circle’s data from global memory with no reuse. This resulted in huge amount of reads.

Finally we reached our Tile-based with a shared-memory circle list version. This combines the best of both approaches; each pixel still has its own owner thread, however for each tile we first

discard the circles that do not touch it and we copy the remaining or kept circles into shared memory so all 256 pixels of the tile can reuse them. This version gave us the 72/72 score.

While we discussed this implementation above but to put it shortly here, We chose tile size 16×16 with 256 threads per block so that the number of pixels in a tile, the number of threads in a block, and the scan width are all the same. That way the same 256 threads without reshuffling can do everything such as test circles, run the scan, compact the survivors, and apply them to pixels. We also tried other numbers but they either had lower occupancy or fewer threads per scan so we figure 16×16 to be the sweet spot for us.

We also thought of combining this lazy approach with the accurate intersection tests, i.e. The starter file provides two circle-vs-box tests; `circleInBoxConservative` (cheap, but with corner false positives) and `circleInBox` (definitive, but more expensive). We thought of running both, lazy first, then the accurate test on the survivors to remove the corner false positives before compacting. But we thought this might add additional overhead instead of speeding it up because the corner false-positive rate is small relative to the true-positive set because the accurate test costs every surviving circle, not just the false positives and the `shadePixelLocal` already does an exact point-in-circle test per pixel, so a false positive that slips through is cheap to discard. That's why we kept just the conservative test.